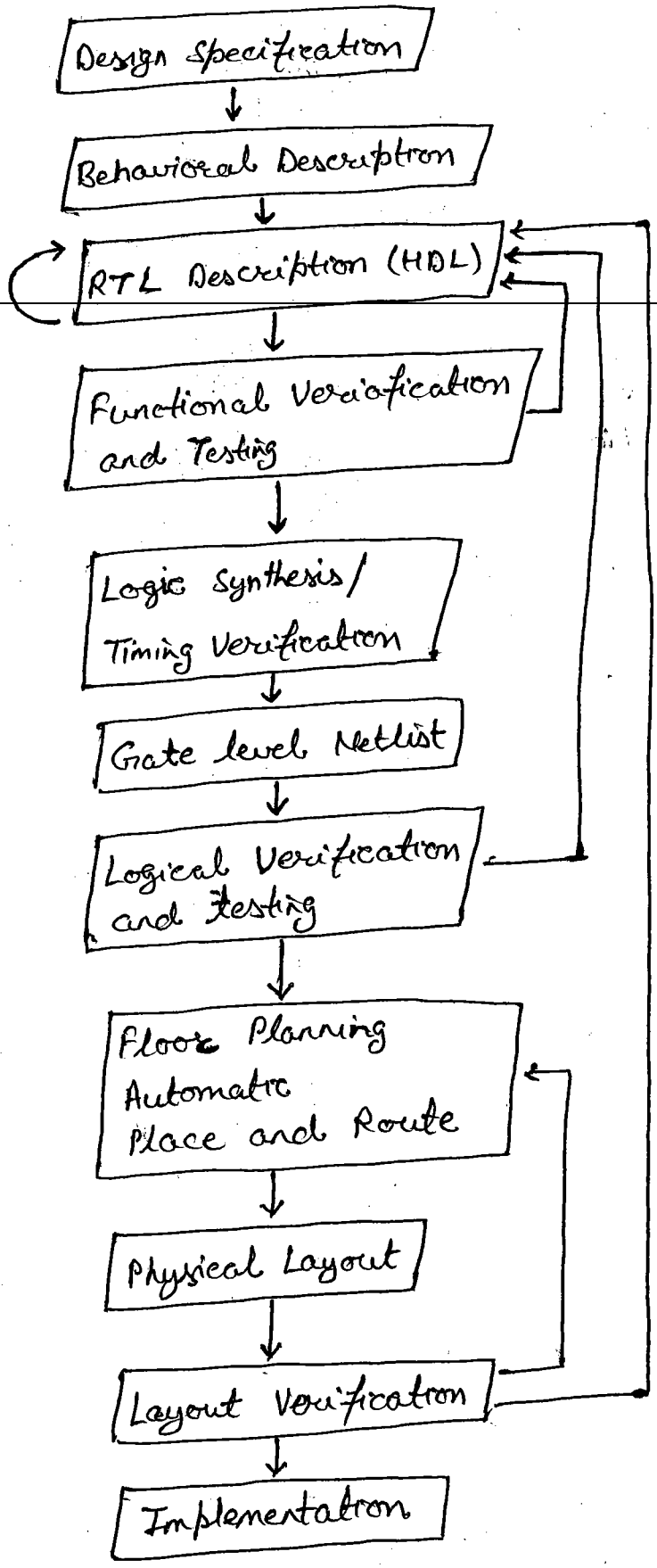
Typical Design Flow



Importance of HDLs:-

- Designs can be described at a very abstract level by use of HDL. Designers can write their RTL description without choosing a specific fabrications technology. Logic synthesis tools can automatically convert the design to any fabrication technology..

- Since designers work at the RTL level, they can optimize and modify the RTL description until it meets the desired functionality.
- A textual description with comments is an easier way to develop and debug circuits. This also provide a concise representation of design, compared to gate-level schematics.

Popularity of Verilog HDL :-

- Verilog HDL is easy to learn and easy to use. It is similar in syntax C programming language.
- A designer can define a hardware model in terms of switches, gates, RTL or behavioral code, also learn only one language stimulus and hierarchical design.
- most popular logic synthesis tool support Verilog HDL.
- All fabrication vendors provide verilog HDL libraries for postlogic synthesis simulation.
- Designers can customize a verilog HDL simulator to their needs with the PLI.

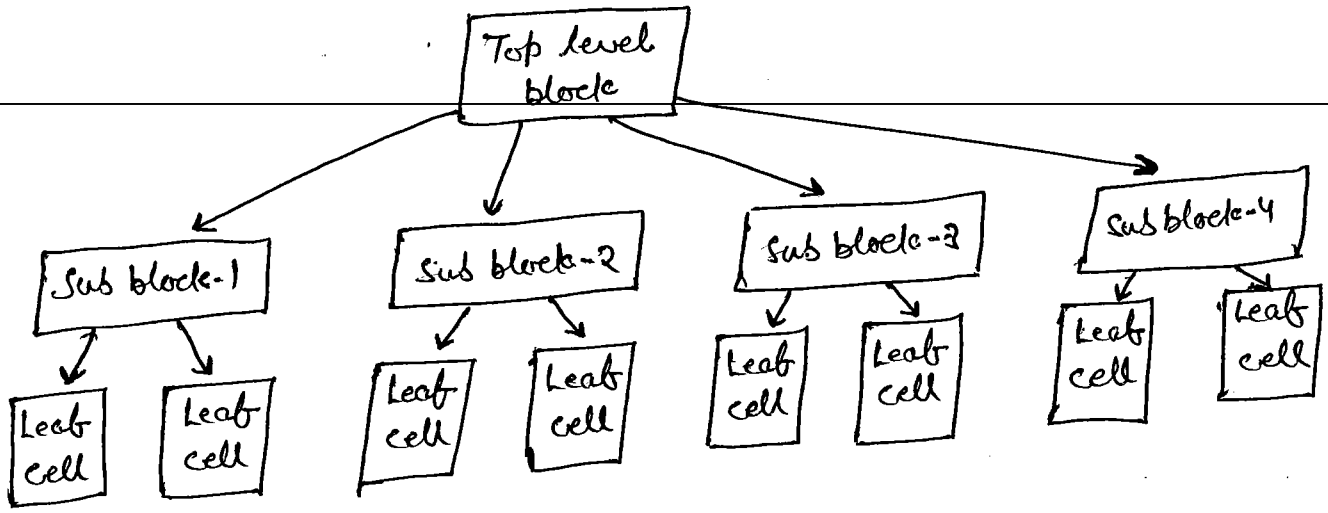
Hierarchical Modeling Concepts

The designers must use a "good" design methodology to do efficient verilog HDL based design.

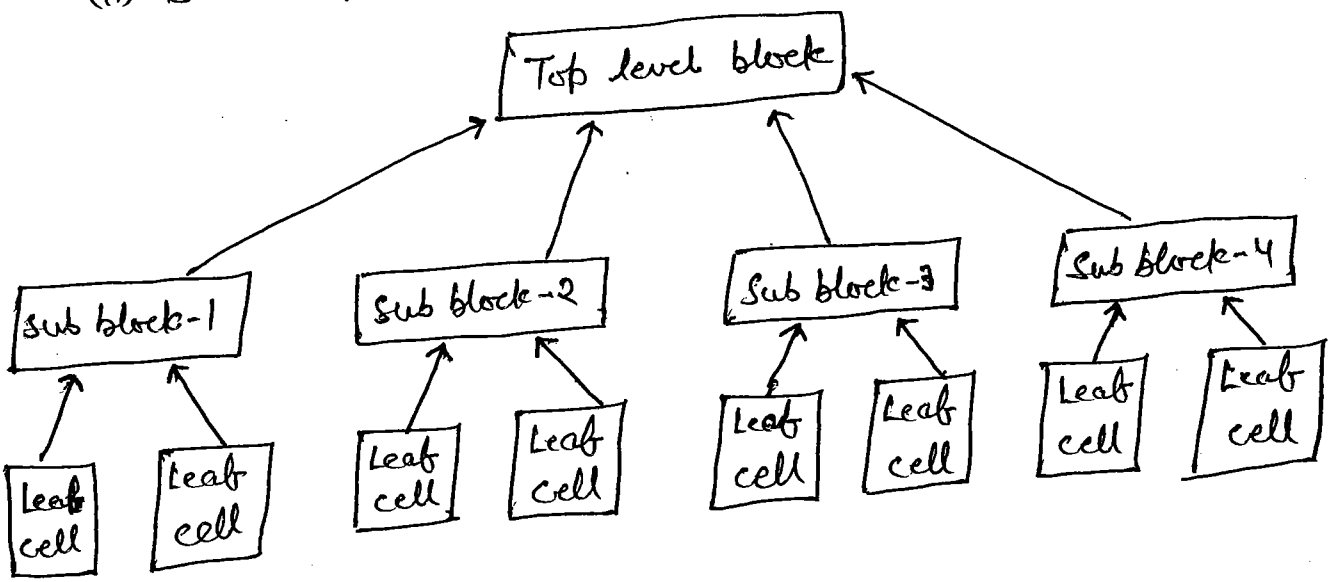
- **Design methodologies!-**

  There are two basic types of digital design methodologies.

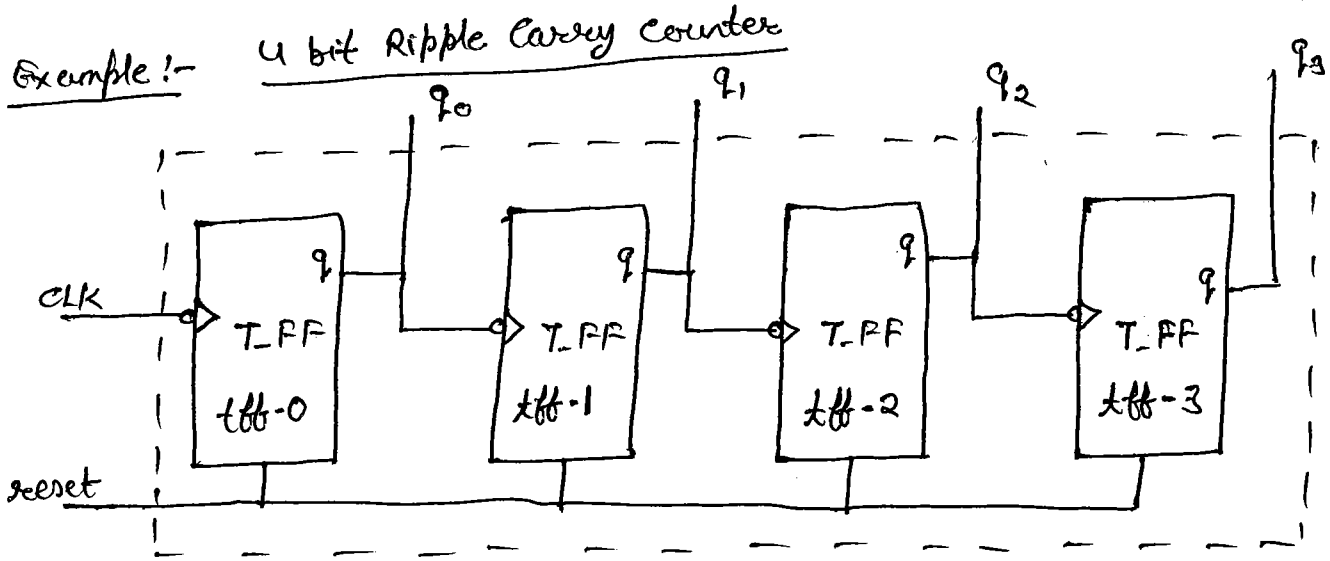
  (i) Top-down design →



(ii) Bottom up design →

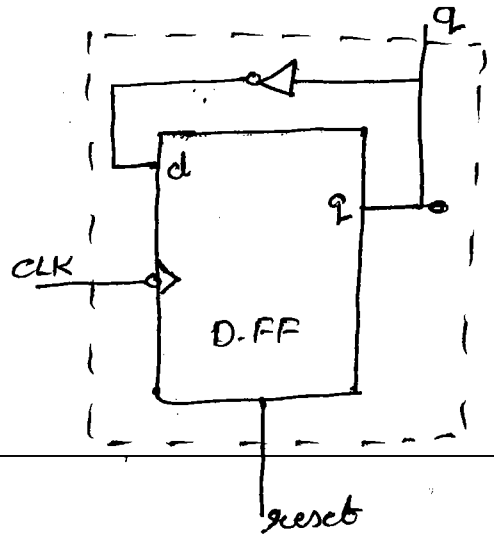


Example!- 4 bit Ripple Carry Counter



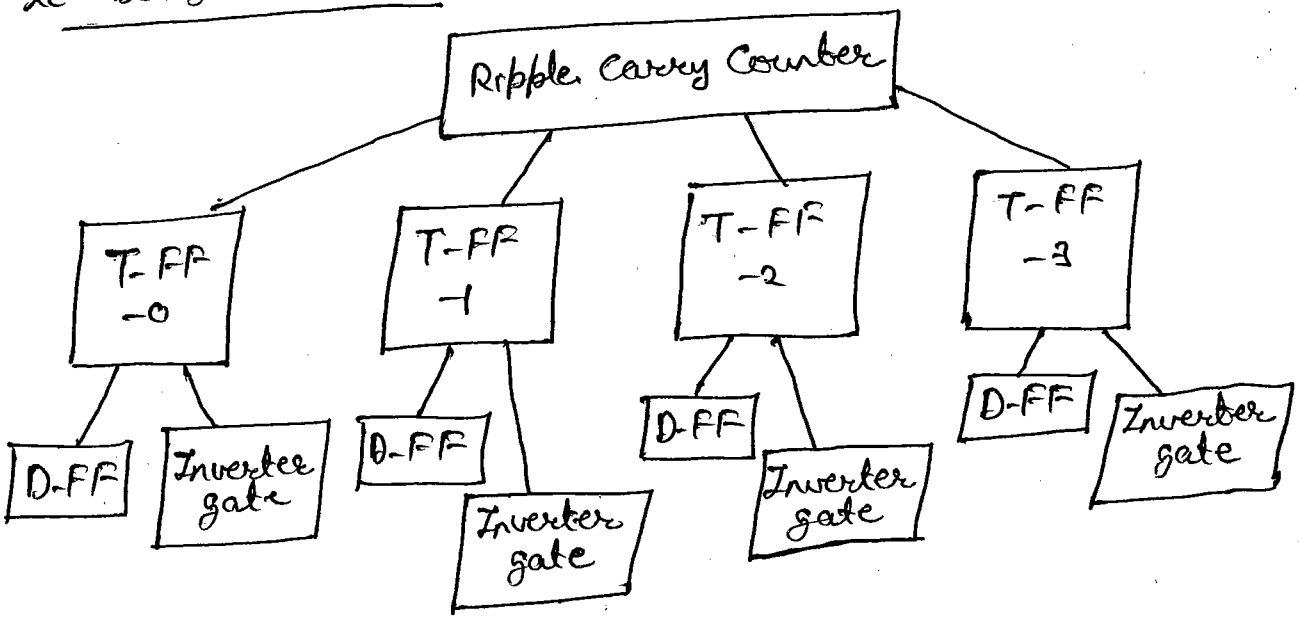
Ripple carry counter

reset	q_n	q_{n+1}
1	1	0
1	0	0
0	0	1
0	1	0
0	0	0



T-flip-flop

Design Hierarchy →



- **Modules:-**

Verilog provides the concept of a module. A module can be an element or a collection of lower-level design blocks. A module provides the necessary functionality to the higher level block through its port interface (input and output) but hides the internal implementation.

Syntax:- module <module_name> (<module_terminal_list>);

```
- - - -
- - - -
<module internals>
- - - -
- - - -
endmodule
```

Internals of each module can be defined at four levels of abstraction, depending on the needs of the design.

- Behavioral or algorithmic level :-

This is the highest level of abstraction provided by Verilog HDL. A module can be implemented in terms of the design algorithm without concern for the hardware implementation details. Designing at this level is very similar to C programming.

- Data flow level :-

The module is designed by specifying the data flow. The designer is aware of how data flows between hardware registers and how the data is processed in the design.

- Gate level :-

The module is implemented in terms of logic gate and interconnection between these gates. Design at this level is similar to describing a design in terms of a gate level logic diagram.

- Switch level :-

A module can be implemented in terms of switches, storage nodes, and the interconnections between them. Design at this level requires knowledge of switch-level implementation details.

- Instances :-

Each object has its own name, variable, parameter, and I/O interface. The process of creating objects from a module template is called instantiation. and the objects are called instances.

example :- The top-level block creates (Ripple carry Counter four instances from the T-FF template. Each T-FF instantiates a D-FF and an inverter gate.

Each instance must be given a unique name.

Basic Concept

Lexical Conventions!-

Convention and concept provide the necessary framework for verilog HDL. Verilog contains a stream of tokens. Tokens can be comments, delimiters, numbers, strings, identifiers and keywords. Verilog HDL is a case-sensitive language. All keywords are in lowercase.

(i) **Whitespace!-**

- Blank spaces (\b)
- Tabs (\t)
- Newlines (\n)

whitespace is ignored by verilog except when it separates tokens. whitespace is not ignored in strings.

(ii) **Comments!-**

Comments can be inserted in the code for reliability and documentation.

- A one-line comments start with "//". Verilog skips from that point to the end of line.
- A multiple-line comments with "/*" and ends with "*/". multiple line cannot be nested.

example!-

- // This is a one-line comment
- /* This is multiple line comment */.
- /* This is /* an illegal */ comment */
- /* This is // a legal comment */

(iii) **Operators!-** Three type -

- Unary
- Binary
- Ternary

Unary operators precedes the operand. Binary operators appear between two operands. Ternary operators have two separate operators with separate three operands.

- a = ~b;
- a = b && c;
- a = b ? c : d;

(iv) Number specification!- Two types -

- Sized Number!- <Size>'<base_format><number>
  ie 4'b1111;

where,
- Size = decimal number

Base format = Decimal - 'd or 'D
Binary - 'b or 'B
Octal - 'o or 'O
Hexadecimal - 'h or 'H

number = specified consecutive digits

- Unsized Number!-
  number that a specified without a ~~[illegible]~~ <Size>
  '<Base format><number>
  ie. 'hc3; '021;

(v) Strings!-
A string is a sequence of characters that are closed by double quotes. The restriction only is that it must be contained on a single line. It cannot be on multiple line.

example!- " Hello verilog world"

(vi) Identifier and Keyword!-
Keyword are in lowercase.
Identifier are made up of alphanumeric characters, the underscore (_) or the dollar sign ($). Identifier are case sensitive. Identifiers start with an alphanumeric character or an underscore. They cannot start with a digit or $ sign

example!- .reg value; // reg is keyword; value is an identifier

.input clk; // input is keyword; clk is an identifier

(vii) Escaped Identifier!-
Escaped identifier begin with the backslash (\) character and end with whitespace.

example!- \a+b-c
my_name

Data Types

(i) **Value Set!-**

Verilog support four values and eight strengths to model the functionality of real hardware.

Value level	Condition in Hardware Circuits
0	Logic zero, false condition
1	Logic one, true condition
X	Unknown logic value
Z	High impedance, floating state

Strength Level	Type	Degree
Supply	Driving	strongest
Strong	Driving	↑
pull	Driving	
large	storage	
weak	Driving	
medium	storage	
small	storage	
highz	High impedance	weakest

(ii) **Nets!-** Net represent connection b/w hardware elements.

Keyword = wire

(iii) **Register!-** Register represent data storage elements. Register retain value until another value is placed onto them.

Keyword = reg.

(iv) **Vectors!-** Net or reg data types can be declared as vectors (multiple bit widths). If bit width is not specified, the default is scaler (1-bit)

Vector can be declared at [high#: Low#] or [low#: high#]

(v) **Integer, Real and Time register!-**

- **Integer!-** Integer store values as signed quantities.

Keyword = integer

- **Real :-** Real number is rounded off to the nearest integer. Real number cannot have range declaration.

  Keyword = real

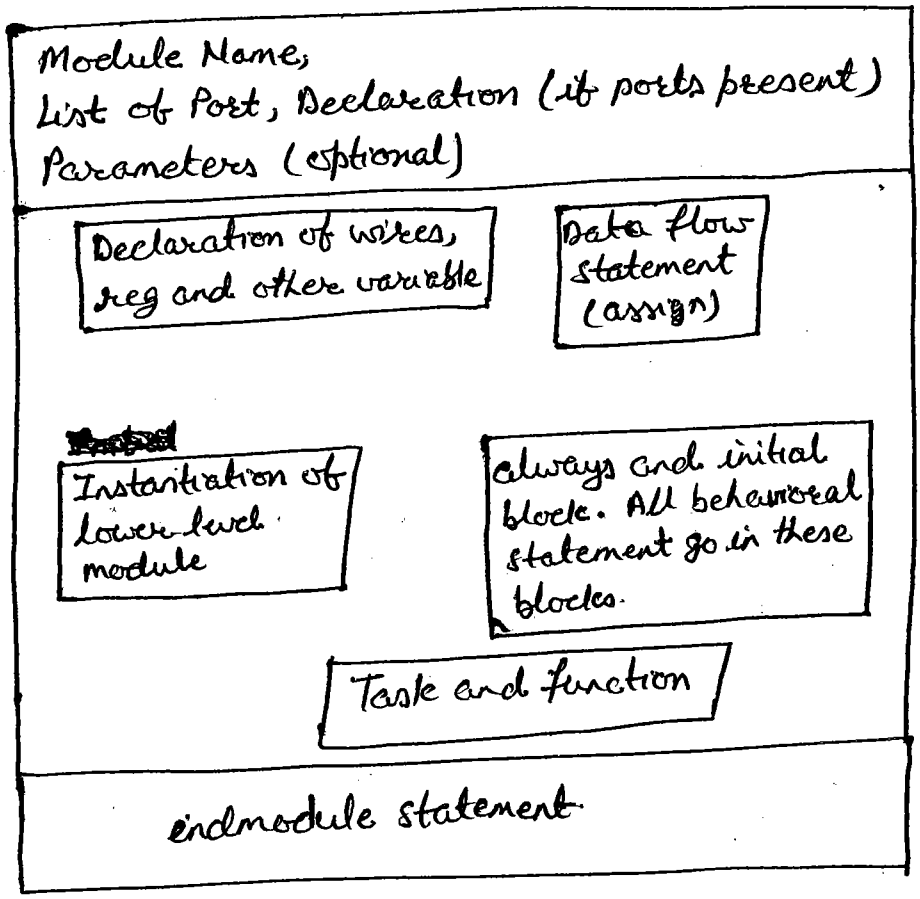
- **Time :-** Verilog simulation is done with respect to simulation time.

  Keyword = $time

(vi) **Array :-** Array are accessed by <array_name>[<subscript>]

ie, integer count[0:7]; // array of 8 count variable.

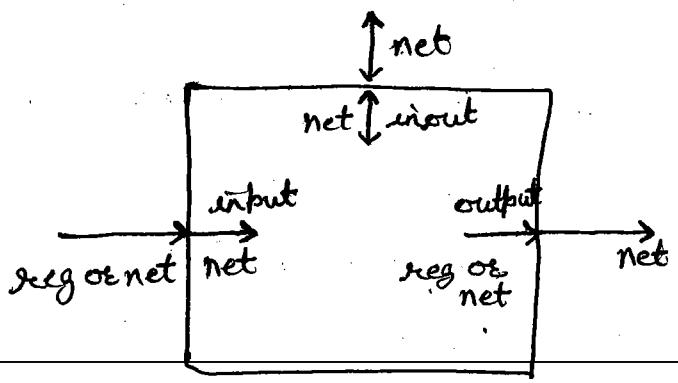
Module



Port

Verilog keyword	Type of Port
input	input port
output	output port
inout	Bidirectional port.

Port Connection Rules



- **Width matching:-** To connect internal and external items of different size when making intermodule port connection.
- **Unconnected Port:-** Verilog allows port to unconnected.

  example:- fulladd4 fa0(sum, , A, B, C_IN);
  // output port C_out is unconnected.

Gate Level Modeling

A logic circuit can be designed by use of logic gates. Verilog support basic logic gates as predefined primitives. There are two classes of basic gates: and/or gates and buf/not gates.

- And/or gates have on scalar output and multiple scalar inputs. (And, Or, Xor, Nand, Nor and Xnor)
- ie,
```
and a1('out, in1, in2);
nand na1(out, in1, in2, in3);
and (out, in1, in2);
```
- Buf/not gate have one scalar input and one or more scalar output. (buf, not).

ie,
```
buf b1(out1; in1);
buf b1(out1, out2, in1);
```

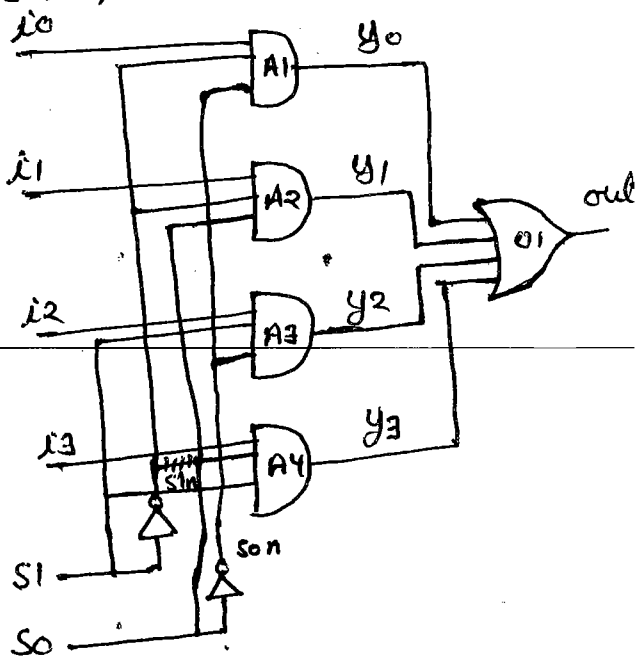
Gate with an control signal on buf and not gates are also,

ie; buf if1, buf if0, not if1, not if0.

ie,
```
buf if1 b1(out, in, ctrl);
not if1 n1(out, in, ctrl);
```

example:- multiplexer

```
module mux_4_1 (out, i0, i1, i2, i3, s0, s1);
input  i0, i1, i2, i3, s0, s1;
output out;
wire   s0n, s1n, y0, y1, y2, y3;
 And  A1 (y0, i0, s1n, s0n);
 And  A2 (y1, i1, s1n, s0);
 And  A3 (y2, i2, s1, s0n);
 And  A4 (y3, i3, s1, s0);
 OR   O1 (out, y0, y1, y2, y3);
endmodule
```



Delay :- and #(5) a1 (out, a, b, c);

Data Flow Modeling

For small circuits, the gate level modeling approach works very well because the number of gates is limited.
In complex design the number of gates is very large. ie data flow modeling provides a powerful way to implement such complex design.

Syntax :- assign [drive_steength] [delay3] list_of_net_assignments;

list_of_net_assignments => net_assignments {, net_assignments}.

net_assignment => net_value = expression

Note :- drive strength is optional.

example :- (i) assign out = i1 & i2;
(ii) assign addr [15:0] = addr1_bits [15:0] ^ addr2_bits[15:0];
(iii) assign {cout, sum[3:0]} = a [3:0] + b [3:0] + C_in;

Implicit Continuous Assignment

example :- ie wire out;
assign out = in1 & in2;

convert to->
wire out = in1 & in2;

Implicit Net Declaration

example :- ie wire i1, i2;
assign out = i1 & i2; } //out is a net.

Expression

Expressions are constructs that combine operators and operands to produce a result.

example:- a ^ b; in1 | in2;

Operators type

Operator Type	Operator Symbol	Operation performed	Number of operands
Arithmetic	*	multiply	two
	/	Divide	two
	+	Add	two
	-	Subtract	two
	%	modulus	two
	**	power (exponent)	two
Logical	!	Logical Negation	one
	&&	Logical And	two
	||	Logical OR	two
Relational	>	Greater than	two
	<	less then	two
	>=	Greater than or equal	two
	<=	less than or equal	two
Equality	==	Equality	two
	!=	Inequality	two
	===	case equality	two
	!==	case inequality.	two
Bitwise	~	Bitwise Negation	one
	&	Bitwise And	two
	|	Bitwise Or	two
	^	Bitwise Xor	two
	^~ or ~^	Bitwise Xnor	two
Reduction	&	Reduction And	one
	~&	Reduction Nand	one
	|	Reduction Or	one
	~|	Reduction Nor	one
	^	Reduction Xor	one
	^~ or ~^	Reduction Xnor	one
Shift	>>	Right Shift	Two
	<<	Left Shift	Two
	>>>	Arithmatic right shift	Two
	<<<	Arithmatic left shift	Two
concatenation	{}	Concatenation	Any Number
Replication	{{3}}	Replication	Any Number
Conditional	?:	Conditional	Three

Operator Precedence

If no parentheses are used to separate parts of expressions, Verilog enforces the following precedence.

Operators	Operator Symbols	Precedence
Unary	+, −, !, ~	Highest precedence
Multiply, Divide, Modulus	*, /, %	
Add, Subtract	+, −	
Shift	<<, >>	
Relational	<, <=, >, >=	
Equality	==, !=, ===, !==	
Reduction	&, ~& ^, ^~ |, ~|	
Logical	&& ||	
Conditional	?:	Lowest precedence

Behavioral Modeling

Verilog provides designers the ability to describe functionality in an algorithms manner. In other words, the designer describes the behavior of the circuit. Thus, behavioral modeling represents the circuit at a very high level of abstraction. Behavioral verilog constructs are similar to C language constructs in many ways.

Structural Procedures

There are two structured procedure statements in verilog: always and initial. All other behavioral statements can appear only inside these structured procedure statement. Each always and initial statement represent a separate activity flow in verilog. and simulation time is 0. These two statement cannot be nested.

- **initial statement:-**

An initial block starts at time 0, executes exactly once during a simulation and then does not execute again. multiple behavioral statement must be grouped, using keyword begin and end.

→ Single statement:-

```
initial
    m = 1'b0;
```

→ multiple statement:-

```
initial
begin #5 a = 1'b1;
    #25 b = 1'b0;
end
```

- always statement:-

This statement execute the statement in the always block continuously in a looping fashion.

---

ie. module clock_gen (output reg clock);

```
initial  clock = 1'b0;
always   #10 clock = ~clock;
initial
         #1000 $finish;
endmodule
```

Conditional Statement

Conditional statements are used for making decisions based upon certain conditions. These condition are used to decide whether or not a statement should be executed. keywords if and else are used for conditional statement.

- if (<expression>) true_statement;
- if (<expressions>) true_statement;
  else false_statement;
- if (<expression>) true_statement1;
  else if (<expression>) true_statement2;
  else if (<expression>) true_statement3;
  else default_statement;

Multiway Branching (case statement)

There were many alternatives, from which one was chosen. The nested if-else-if can become unwidely if there are two many alternatives.

- case (expression)
  alternative1 : statement1;
  alternative2 : statement2;
  alternative3 : statement3;
  - - - -
  default : default_statement;

example:-
```
case (alu-control)
    2'd0 : y = X * Z;
    2'd1 : y = X + Z;
    2'd2 : y = X - Z;
  default : $display ("Invalid ALU control signal");
endcase
```

Loops statement

All (while, for, repeat, forever) looping statement are to be appear only inside an initial or always block.

- **While loop:-**

The while loop executes until the while expression is not true. If multiple statement are to be execute in the loop, they must be grouped typecally using keywords begin and end.

example:-
```
integer count;
initial
begin
        count = 0;
while (count < 128)
begin
        $display ("count = %d", count);
end
end.
```

- **For loop:-** for loop contains
  (i) An initial condition.
  (ii) A check to see if the terminating condition is true.
  (iii) A procedural assignment to change value of the control variable.

example:-
```
integer count;
initial
    for (count = 0; count < 128; count = count + 1)
        $display ("count = %d", count);
```

- **Repeat loop:-** The repeat construct executes the loop a fixed number of times. It contain a number, which can be a constant, a variable or a signal value.

example:-
```
integer count;
initial
 begin
        count = 0;
        repeat (128)
        begin $display ("count = %d", count);
        end
end.
```

- **Forever loop:-** The loop does not contain any expression and executes forever until the $finish task is encountered. It is used in conjunction with timing control constructs.

example:-
```
reg clock;
initial
begin
    clock = 1'b0;
    forever #10 clock = ~clock;
end
```

Switch Level Modeling

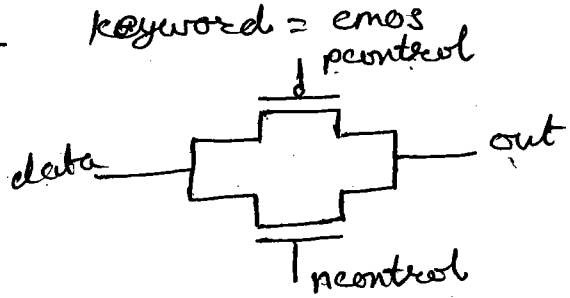
- **mos switches:-** keyowred = nmos & pmos


```
nmos  n1 (out, data, control);
pmos  p1 (out, data, control);
```

nmos (data \ control)	0	1	X	Z
0	Z	0	L	L
1	Z	1	H	H
X	Z	X	X	X
Z	Z	Z	Z	Z

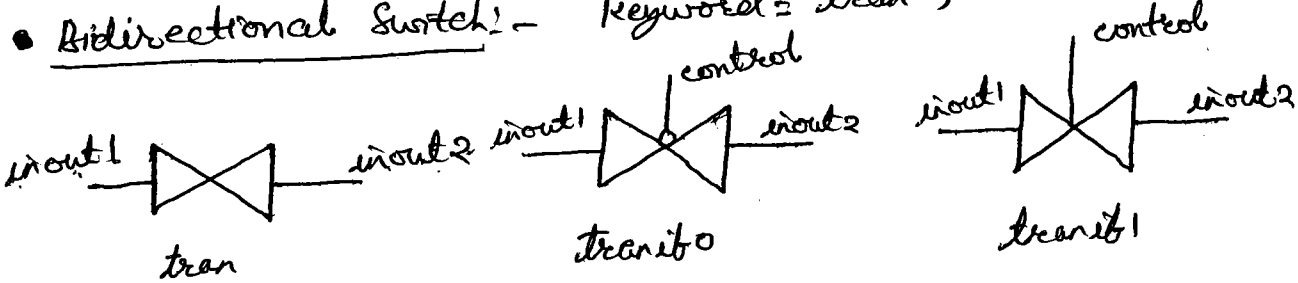
pmos (data \ control)	0	1	X	Z
0	0	Z	L	L
1	1	Z	H	H
X	X	Z	X	X
Z	Z	Z	Z	Z

- **cmos switch:-** keyword = cmos




```
cmos  c1('out, data, ncontrol, pcontrol);
cmos (out, data, ncontrol, pcontrol);
```

- **Bidirectional Switch:-** keyword = tran, tranif0, tranif1.



```
tran t1 (inout1, inout2);
tranif0 t2 (inout1, inout2, control);
tranif1 t3 (inout1, inout2, control);
```

- **Power and Ground:-**

Power (Vdd, logic 1) Ground (V_{SS}, logic 0)

Keyword = supply1, supply0

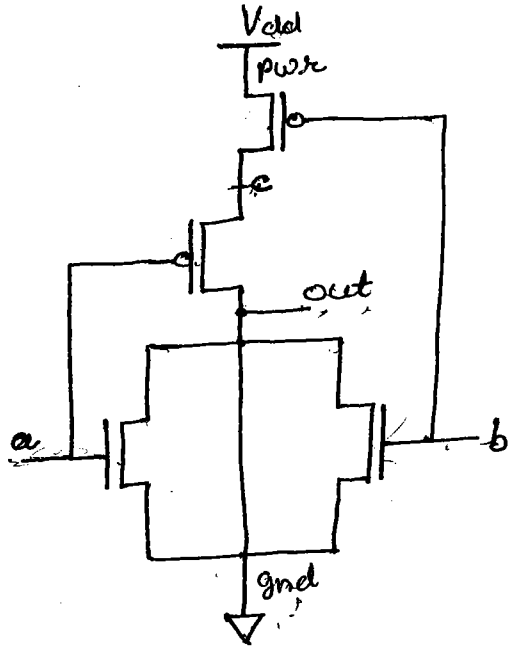
---

example:-

```
supply1 Vdd;
supply0 Vss;          (Vss = gnd)
assign a = Vdd;
assign b = Vss;
```

Example:- CMOS NOR Gate:-

```
module my-nor (out, a, b);
output out;
input a, b;
wire c;
supply1 • pwr;
supply0 gnd;
pmos (c, pwr, b);
pmos (out, c, a);
nmos (out, gnd, a);
nmos (out, gnd, b);
endmodule
```



UDP (User Define Primitive)

UDP exactly like gate-level primitives. Verilog provide the ability to primitives are self-contained and do not instantiate other modules.

Syntax:-

```
primitive <udp-name> (<output_terminal-
                      name> (only one allowed),
                      <input_terminals-name>);

output  <output_terminal_name>;
input   <input_terminals-name>;
reg     <output_terminal_name>;
        (optional, only for sequential UDP)
initial <output_terminal_name> = <value>;
table
        <table_entries>;
endtable
endprimitive
```

Table entries:-

```
<input1> <input2> --- <inputn> : <output>;
(ie repeat m times)

<input1> <input2> ---- <inputn> : <current_state>
                                : <next_state>;
(ie repeat m times).
```

UDP Rules

(i) UDP can take only scalar input terminals (1 bit). Multiple input terminals are permitted.
(ii) UDP can have only one scalar output terminal (1 bit). Multiple output terminals are not allowed.
(iii) The output terminal must also be declared as a reg.
(iv) The inputs are declared with the keyword input.
(v) The state in a sequential UDP can be initialized with an initial statement.
(vi) The state table entries can contain values 0, 1, x. UDP do not handle z values.
(vii) UDP are defined at the same level as modules. UDP cannot be defined inside module.
(viii) UDP do not support inout ports.